

The `update()` method ❸ must be called every time through the main loop. If the timer is running and the appropriate amount of time has elapsed, this method returns `True`. In any other call, this method returns `False`.

If a `Timer` is running, calling `getTime()` ❹ returns how much time has elapsed for that `Timer`. You can call the `stop()` method ❺ to immediately stop the `Timer`.

We can now rewrite the timer demonstration program shown in [Figure 13-1](#) to use this `Timer` class from the `pyghelpers` package. [Listing 13-3](#) shows how we use a `Timer` object in the code.

File: `TimerObjectExamples/SimpleTimerExample.py`

```
# Simple timer example

--- snip ---

❶ oTimer = pyghelpers.Timer(TIMER_LENGTH) # create a Timer object

# 6 - Loop forever
while True:

    # 7 - Check for and handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

        if startButton.handleEvent(event):
            ❷ oTimer.start() # start the timer
            startButton.disable()
            timerMessage.show()
            print('Starting timer')

        if clickMeButton.handleEvent(event):
            print('Other button was clicked')

    # 8 - Do any "per frame" actions
    ❸ if oTimer.update(): # True here means timer has ended
        startButton.enable()
        timerMessage.hide()
        print('Timer ended')
```